

GPIO Driver Analysis

Introduction

The GPIO (General Purpose Input/Output) driver is used to configure and control the digital I/O pins of the STM32 microcontroller. It provides a hardware abstraction layer that simplifies register-level programming through user-friendly APIs.

Initialization

GPIO initialization is performed using the `GPIO_Init()` function. During initialization, the following parameters are configured:

- * Pin Number
- * Pin Mode (Input, Output, Alternate Function, Analog)
- * Output Speed
- * Pull-up/Pull-down Configuration
- * Output Type

The driver first enables the clock for the corresponding GPIO peripheral and then configures the required registers.

Data Input and Output Operations

The GPIO driver provides the following APIs:

- * `GPIO_ReadFromInputPin()` – Reads the status of a GPIO pin.
- * `GPIO_ReadFromInputPort()` – Reads the entire GPIO port.
- * `GPIO_WriteToOutputPin()` – Writes HIGH or LOW to a GPIO pin.
- * `GPIO_WriteToOutputPort()` – Writes data to the entire port.
- * `GPIO_ToggleOutputPin()` – Toggles the current state of a GPIO pin.

Design Features

- * Hides low-level register operations.
- * Provides reusable and portable APIs.
- * Supports simple polling-based I/O operations.
- * Easy integration with application code.

Practical Example

In this project, the GPIO driver was used to blink the onboard LED of the STM32 Nucleo board. The `GPIO_ToggleOutputPin()` API was called periodically to generate the blinking effect.

Conclusion

The GPIO driver simplifies digital input/output operations and enables efficient control of LEDs, buttons, and other external devices.

VEDIO DEMONSTRATION LINK



GPIO.MOV

USART Driver Analysis

Introduction

The USART (Universal Synchronous Asynchronous Receiver Transmitter) driver is used for serial communication between the STM32 microcontroller and external devices such as PCs, sensors, and communication modules.

Initialization

USART initialization is performed using the `USART_Init()` function. The following parameters are configured:

- * Baud Rate
- * Word Length
- * Stop Bits
- * Parity Control
- * Hardware Flow Control

The driver enables the USART peripheral clock and configures the necessary communication registers.

Data Transfer Operations

The USART driver provides the following APIs:

- * USART_SendData() – Transmits data over the serial interface.
- * USART_ReceiveData() – Receives data from the serial interface.
- * USART_PeripheralControl() – Enables or disables the USART peripheral.

Design Features

- * Simplifies serial communication setup.
- * Provides separate APIs for transmission and reception.
- * Abstracts hardware-specific details.
- * Suitable for debugging and communication applications.

Practical Example

In this project, USART communication was demonstrated using a serial terminal application (PuTTY). Data transmitted from the STM32 board was displayed successfully on the terminal.

Conclusion

The USART driver provides a reliable and efficient method for serial communication and is widely used for debugging, monitoring, and device-to-device communication.

VEDIO DEMONSTRATION LINK



USART .mov

I2C Driver Analysis

Introduction

The I2C (Inter-Integrated Circuit) driver is used for communication between the STM32 microcontroller and peripheral devices such as temperature sensors, EEPROMs, RTC modules, and display modules. I2C is a two-wire communication protocol that uses Serial Data (SDA) and Serial Clock (SCL) lines.

Initialization

The I2C peripheral is initialized using the `I2C_Init()` function. During initialization, the following parameters are configured:

- * Device Address
- * SCL Clock Speed
- * Fast Mode Duty Cycle

- * Acknowledge Control
- * Peripheral Enable Settings

The driver enables the I2C peripheral clock and configures the required registers for communication.

Data Transfer Operations

The I2C driver provides the following APIs:

- * I2C_MasterSendData() – Sends data from the STM32 master to a slave device.
- * I2C_MasterReceiveData() – Receives data from a slave device.
- * I2C_PeripheralControl() – Enables or disables the I2C peripheral.
- * I2C_ManageAcking() – Controls the ACK mechanism during communication.

The driver handles important I2C operations such as START condition generation, address transmission, data transfer, acknowledgment management, and STOP condition generation.

Design Features

- * Simplifies communication with I2C devices.

- * Hides complex register-level operations.
- * Supports master-mode communication.
- * Provides reusable APIs for various sensor and memory devices.

Practical Example

In this project, the I2C driver can be used to communicate with an external temperature sensor or EEPROM. The microcontroller sends commands and receives data through the I2C bus using the provided APIs.

Conclusion

The I2C driver provides an efficient method for interfacing STM32 microcontrollers with sensors and other low-speed peripherals. It simplifies communication while maintaining reliability and flexibility.

VEDIO DEMONSTRATION LINK



I2c.mov

SPI Driver Analysis

Introduction

The SPI (Serial Peripheral Interface) driver is used for high-speed communication between the STM32 microcontroller and external devices such as displays, sensors, memory modules, and communication chips. SPI is a full-duplex communication protocol that uses separate data lines for transmission and reception.

Initialization

SPI initialization is performed using the `SPI_Init()` function. The following parameters are configured:

- * Device Mode (Master/Slave)
- * Bus Configuration
- * Clock Speed
- * Data Frame Format
- * Clock Polarity (CPOL)
- * Clock Phase (CPHA)
- * Software Slave Management (SSM)

The driver enables the SPI peripheral clock and configures the necessary communication registers.

Data Transfer Operations

The SPI driver provides the following APIs:

- * `SPI_SendData()` – Transmits data to an SPI slave device.
- * `SPI_ReceiveData()` – Receives data from an SPI slave device.
- * `SPI_PeripheralControl()` – Enables or disables the SPI peripheral.
- * `SPI_SSIConfig()` – Configures internal slave select settings. The driver manages data transfer through SPI data registers and status flags while ensuring proper synchronization between master and slave devices.

Design Features

- * Supports full-duplex communication.
- * Provides high-speed data transfer.
- * Abstracts low-level register configurations.
- * Suitable for displays, sensors, and memory devices.

Practical Example

In this project, the SPI driver can be used to transmit test data to an SPI device such as an OLED display, LCD module, or external sensor. Data is sent using the SPI_SendData() API and verified through the connected peripheral.

Conclusion

The SPI driver enables fast and reliable communication between the STM32 microcontroller and external devices. Its modular API design makes it easy to integrate SPI peripherals into embedded applications.

VEDIO DEMONSTRATION LINK



Spi.MOV